

LẬP TRÌNH CHO TRÍ TUỆ NHÂN TẠO

Giảng viên: Nguyễn Ngọc Giang

Nội dung bài trước

- Biến, khai báo biến, ghi chú và khối lệnh
 - Biến và cách khai báo biến
 - Cách xóa biến, kiểm tra vùng lưu trữ của biến
 - Ghi chú, cách ghi chú và tại sao cần ghi chú
 - Khối lệnh
- Nhập dữ liệu và xuất dữ liệu
- Các kiểu dữ liệu cơ bản và các phép toán liên quan
 - Các kiểu dữ liệu số cơ bản và các toán tử số cơ bản
 - Các toán tử gán, logic, so sánh, bit
 - Độ ưu tiên toán tử
- Điều khiển rẽ nhánh
 - Biểu thức if
 - Biểu thức if,else
 - Biểu thức if,...elif,....else
- Hàm
 - Định nghĩa và cấu trúc hàm
 - Cách xây dựng và gọi hàm

Nội dung hôm nay

1. Vòng lặp While
2. Vòng lặp For
3. Xử lý chuỗi
4. List (Danh sách)
5. Tuple
6. Range

Phần 1

Vòng lặp “while”

Vòng lặp while

while expression:

while-block

while expression:

#while-block-1

continue

#while-block-2

while expression:

while-block

else:

else-block

- Vòng **while** thực hiện lặp lại khối lệnh chừng nào biểu thức điều kiện còn đúng
 - Phát biểu **continue** trong khối lệnh sẽ ngắt khối lệnh hiện tại và bắt đầu một vòng lặp mới
 - Phát biểu **break** sẽ kết thúc vòng lặp ngay lập tức
- Khối **else** sẽ được thực hiện sau khi toàn bộ vòng lặp đã chạy xong, không bắt buộc phải có khối này
 - Khối này sẽ không chạy nếu vòng lặp bị “break”

Vòng lặp while đơn giản

```
# có 10 triệu đồng, gửi ngân hàng với lãi suất 5,1% hàng năm
# tính xem sau bao nhiêu năm thì bạn có ít nhất 50 triệu
# cách giải sử dụng vòng lặp
so_tien = 1e7
lai_suat = 5.1/100
so_nam = 0
# chừng nào số tiền chưa đủ 50 triệu thì gửi thêm 1 năm nữa
while so_tien < 5e7:
    so_nam += 1
    so_tien = so_tien * (1 + lai_suat)
    print("Số tiền sau", so_nam, "năm:", so_tien)
# in kết quả
print("Sau", so_nam, "bạn sẽ có ít nhất 50 triệu.")
```

Vòng lặp while kết hợp điều kiện if

In ra các số tự nhiên chia hết cho 7 nhỏ hơn 1000

```
n = 0
```

```
while n < 1000:
```

```
    if (n % 7) == 0:
```

```
        print(n)
```

```
    n += 1
```

Tính tổng các số nhỏ hơn 1000 và không chia hết cho 3

```
t = 0
```

```
n = 0
```

```
while n < 1000:
```

```
    if (n % 3) != 0:
```

```
        t = t + n
```

```
    n += 1
```

```
print(t)
```

Vòng lặp while với break

```
# bài tập buổi trước: kiểm tra xem một số dương N có phải số  
# fibonacci hay không?  
n = int(input("Nhập số dương N: "))  
a, b = 0, 1                # kiểu gán trong python: a = 0, b = 1  
while b != n:  
    if b > n:                # nếu b vượt quá n thì dừng  
        break  
    a, b = b, a + b          # a = b, b = a + b  
print('Fibonacci' if b == n else 'Non-fibonacci')
```


Vòng lặp while với continue

```
# tính tổng các số fibonacci chia hết cho 3 nhỏ hơn N
n = int(input("Nhập số dương N: "))
tong, a, b = 0, 0, 1
while b < n:
    a, b = b, a + b
    if 0 != a % 3:                # bỏ qua nếu không chia hết cho 3
        continue
    tong += a
print('Tổng là:', tong)
```

Vòng lặp while sử dụng else

```
# nhập số n và kiểm tra xem nó có phải số nguyên tố hay không
# chú ý: nếu không sử dụng else, ta sẽ phải khai báo thêm
# biến phụ và chương trình dài hơn vài dòng

n = int(input("Nhập số N: "))
x = 2
while x < n:
    if (n % x) == 0:
        print("N không phải số nguyên tố")
        break;
    x = x + 1
else:
    print("N là số nguyên tố")
```

Phần 2

Vòng lặp “for”

Vòng lặp for duyệt một danh sách

- Cú pháp:

```
for <biến> in <danh-sách>:
```

```
    # khối for
```

```
else
```

```
    # khối else
```

- Vòng **for** cho phép sử dụng một <biến> lần lượt duyệt các giá trị trong <danh-sách>
- Tương tự như while, có thể sử dụng **break** và **continue**
- Khối **else** thực hiện sau khi toàn bộ vòng lặp đã chạy xong
 - Khối này sẽ không chạy nếu vòng lặp bị “break”
 - Không bắt buộc phải có khối này
 - Cách làm việc tương tự như ở vòng lặp while

Vòng lặp for duyệt một danh sách

```
X = ['chó', 'mèo', 'lợn', 'gà']  
# In ra các loài vật trong danh sách  
for w in X:  
    print(w)  
# In ra các loại vật, ngoại trừ loài 'mèo'  
for x in X:  
    if x == 'mèo':  
        continue  
    print(x)  
# In ra các loại vật, nếu gặp loài 'mèo' thì dừng luôn  
for z in X:  
    if z == 'mèo':  
        break  
    print(z)
```

Vòng lặp for duyệt một miền số nguyên

- Cú pháp vòng **for** rất phù hợp với việc duyệt một tập hợp ít phần tử
 - Vì ta phải liệt kê mọi phần tử trong tập
- Nhưng nếu muốn duyệt tập rất nhiều phần tử thì sao?
 - Chẳng hạn muốn duyệt các số nguyên từ 1 đến 1.000.000?
- Python cung cấp hàm **range** để tạo một dãy số:
begin: Giá trị bắt đầu
end: Giá trị cuối
step: Bước nhảy

`range( begin, end, step )`

Vòng lặp for duyệt một miền số nguyên

- Python cung cấp hàm `range` để tạo một dãy số:
 - Hàm `range(n)`: tạo dãy số nguyên từ 0 đến $n-1$
 - Hàm `range(n, m)`: tạo dãy số nguyên từ n đến $m-1$
 - Hàm `range(n, m, k)`: tạo dãy số nguyên từ n đến trước m với bước nhảy k (một lần giá trị tăng k đơn vị)
 - Chú ý: giá trị k có thể âm, trong trường hợp này dãy số sinh ra sẽ giảm dần

Ví dụ cách hoạt động của `range`:

- `range(10)` → 0; 1; 2; 3; 4; 5; 6; 7; 8; 9
- `range(1, 10)` → 1; 2; 3; 4; 5; 6; 7; 8; 9
- `range(1, 10, 2)` → 1; 3; 5; 7; 9
- `range(10, 0, -1)` → 10; 9; 8; 7; 6; 5; 4; 3; 2; 1
- `range(10, 0, -2)` → 10; 8; 6; 4; 2
- `range(2, 11, 2)` → 2; 4; 6; 8; 10

Vòng lặp for duyệt một miền số nguyên

Các Ví dụ về for:

```
for n in range(10) : —————> 0 1 2 3 4 5 6 7 8 9  
    print(n, end=' ')
```

```
for n in range(1, 10) : —————> 1 2 3 4 5 6 7 8 9  
    print(n, end=' ')
```

```
for n in range(1, 10, 2) : —————> 1 3 5 7 9  
    print(n, end=' ')
```

```
for n in range(10, 0, -1) : —————> 10 9 8 7 6 5 4 3 2 1  
    print(n, end=' ')
```

```
for n in range(10, 0, -2) : —————> 10 8 6 4 2  
    print(n, end=' ')
```

```
for n in range(2, 11, 2) : —————> 2 4 6 8 10  
    print(n, end=' ')
```


Vòng lặp for duyệt một miền số nguyên

Trường hợp một khoảng số khá lớn, không thể liệt kê được

Ta sử dụng hàm range để tạo ra khoảng số

In các số từ 10 đến 19: khoảng 10 đến 20, bước nhảy 1

```
for d in range(10, 20):
```

```
    print(d)
```

In các số từ 20 đến 11: khoảng 20 đến 10, bước nhảy -1

```
for d in range(20, 10, -1):
```

```
    print(d)
```

In các số lẻ từ 1 đến 100: khoảng 1 đến 100, bước nhảy 2

```
for d in range(1, 101, 2):
```

```
    print(d)
```

Một vài ví dụ

Ví dụ: Viết chương trình vòng lặp vĩnh cửu cho phép phần mềm chạy liên tục, khi nào hỏi thoát mới thoát phần mềm:

```
while True:
    a=int(input("Nhập giá trị:"))
    print("Giá trị bạn nhập ",a)
    s=input("Tiếp tục phần mềm không? (c/k) :")
    if s=="c":
        break
print("BYE!")
```

Một vài ví dụ

Ví dụ: Tính tổng các chữ số lẻ từ 1->15, ngoại trừ số 3 và số 11

```
sum=0
for n in range(1,16,2):
    if n is 3 or n is 11:
        continue
    sum+=n
print(sum)
```

Một vài ví dụ

Ví dụ: Nhập vào danh sách 5 số dương và đưa ra kết quả trung bình của danh sách đó. Nếu số nhập vào < 0 thì kết thúc chương trình.

```
count = sum = 0
print('Nhập danh sách các số dương để tính trung')
while count < 5:
    val = float(input('Nhập số: '))
    if val < 0:
        print('Số 0 sai quy tắc, thoát phần mềm')
        break
    count += 1
    sum += val
else:
    print('Trung Bình =', sum/count)
```

Một vài ví dụ

Ví dụ: Vòng for lồng nhau, Python cũng tương tự các ngôn ngữ khác, có thể viết các vòng for lồng nhau

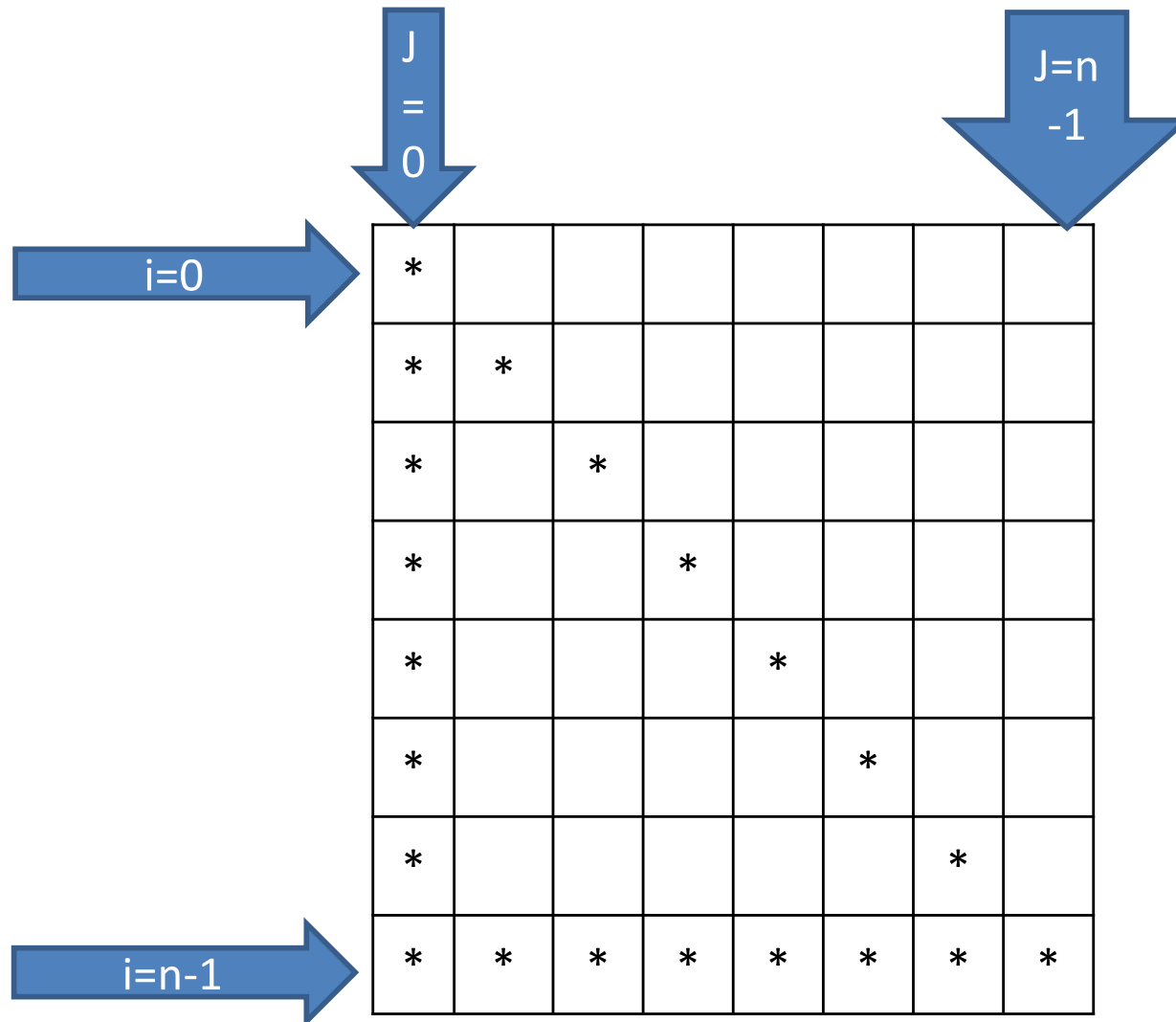
```
n=int(input("Nhập chiều cao:"))
for i in range(n):
    for j in range(n):
        if j==0 or i==j or j==n-1:
            print("*",end=' ')
        else:
            print(" ",end=' ')
    print()
```



Nhập chiều cao:7

```
*      *
**     *
* *    *
*  *   *
*    * *
*     **
*      *
```

Một vài ví dụ



Bài tập

Bài 1: Viết hàm `is_prime` kiểm tra xem N có phải số nguyên tố hay không?

Bài 2: Viết chương trình nhập hai số A và B , in ra tất cả các số nguyên tố nằm trong khoảng $[A, B]$

Bài 3: Nhập 2 số nguyên dương A và B , tính và in ra màn hình ước số chung lớn nhất và bội số chung nhỏ nhất của 2 số đó.

Bài 4: Viết chương trình cho phép người dùng nhập vào liên tiếp một dãy số tự nhiên (không biết trước độ dài). Việc nhập dãy sẽ kết thúc khi người dùng nhập một số âm hoặc số 0.

Sau khi kết thúc, in ra màn hình ước số chung lớn nhất và bội số chung nhỏ nhất của tất cả các số vừa nhập vào.

Bài 5: Viết chương trình cho phép người dùng nhập vào liên tiếp một dãy số bất kì (không biết trước độ dài). Việc nhập dãy sẽ kết thúc khi người dùng nhập vào số 0.

Sau khi kết thúc, in ra màn hình số lượng số đã nhập, giá trị nhỏ nhất và giá trị lớn nhất của dãy số vừa nhập.

Phần 3

String (chuỗi)

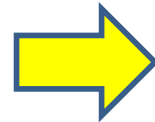
Kiểu chuỗi

- Một chuỗi được xem như một hàng (tuple) các chuỗi con độ dài 1
 - Trong python không có khái niệm kiểu kí tự (character)
 - Nội dung của chuỗi không thay đổi được, khi ghép thêm nội dung vào chuỗi thực chất là tạo ra chuỗi mới
 - Chuỗi trong python hỗ trợ các kí tự unicode, vì vậy có thể sử dụng tiếng Việt (cũng như tiếng Hán, tiếng Nhật,...) thoải mái
- Hàm `len(s)` trả về độ dài (số kí tự) của s, hàm trả về 0 nếu chuỗi rỗng
- Phép toán với chuỗi:
 - Phép nối chuỗi (+): `s = "Good" + " " + "Morning!"`
 - Phép nhân bản (*): `s = "AB" * 3` # ABABAB
 - Kiểm tra nội dung: `s in '1ABABABCD'` # True

Kiểu chuỗi

Chuỗi là tập các ký tự nằm trong nháy đơn hoặc nháy đôi, hoặc 3 nháy đơn hoặc 3 nháy đôi. Chuỗi rất quan trọng trong mọi ngôn ngữ, hầu hết ta đều gặp xử lý chuỗi

```
1 s1='Hello Simon'
2 s2="Hello David"
3 s3=""
4 Quanh năm buôn bán ở mom sông
5 nuôi đủ năm con với một chồng
6 lặn lội thân cò khi quãng vắng
7 eo sèo mặt nước buổi đò đông
8 ""
9 s4='''Cha mẹ thói đời ăn ở bạc
10 có chồng hờ hững cũng như không'''
11 print(s1)
12 print(s2)
13 print(s3)
14 print(s4)
```



```
Hello Simon
Hello David

Quanh năm buôn bán ở mom sông
nuôi đủ năm con với một chồng
    lặn lội thân cò khi quãng vắng
    eo sèo mặt nước buổi đò đông

Cha mẹ thói đời ăn ở bạc
    có chồng hờ hững cũng như không
```

Ví dụ về phép nối chuỗi

```
a = 'Hello'
b = 'World'
print(a + b)                # "HelloWorld"
c = 'Hello' 'Kitty'
print(c)                    # "HelloKitty"
d = '1'
  '2'
  '3'
print(d)                    # "1"
e = ('1'
     '2'
     '3')
print(e)                    # "123"
print(d + '23')             # "123"
print(e '23')               # lỗi
```

Ví dụ về phép nhân bản và kiểm tra nội dung

```
print('abc' * 3)           # 'abcabcabc'
print(5 * '10')           # '1010101010'
print('ab' + 'cd' * 3)    # 'abcdcdcd'

print('bcd' in 'abcdef')   # True
print('bcd' in 'abc' * 3)  # False
print('abc' not in 'ab' + 'cd') # False
```

Phép so sánh giữa 2 chuỗi

- Phép so sánh chuỗi trong Python sử dụng trật tự từ điển, khi hai chuỗi A và B được so sánh với nhau, Python thực hiện theo quy tắc sau:
 - So sánh lần lượt từng cặp từ trái qua phải, tương ứng theo từng ký tự
 - Ký tự của chuỗi nào lớn hơn thì chuỗi tương ứng lớn hơn
 - Nếu hai ký tự giống nhau thì chuyển sang cặp tiếp theo
 - Nếu một chuỗi còn ký tự nhưng chuỗi kia đã hết thì chuỗi đã hết nhỏ hơn

Chỉ mục trong chuỗi

- Các phần tử (các chữ) trong chuỗi được đánh số thứ tự và có thể truy cập vào từng phần tử theo chỉ số
- Python duy trì 2 cách đánh chỉ mục khác nhau:
 - Từ trái qua phải: chỉ số đánh từ 0 tăng dần đến cuối chuỗi
 - Từ phải qua trái: chỉ số đánh từ -1 giảm dần về đầu chuỗi
 - Hai cách đánh chỉ mục này có thể sử dụng lẫn lộn với nhau, chẳng hạn: lấy từ vị trí 1 đến vị trí -2 của chuỗi ĐHTHUYLOI ta sẽ được chuỗi HTHUYL

Đ	H	T	H	U	Y	L	O	I
0	1	2	3	4	5	6	7	8
-9	-8	-7	-6	-5	-4	-3	-2	-1

Cắt chuỗi

- Dựa trên chỉ mục, phép cắt chuỗi cho phép lấy nội dung bên trong của chuỗi bằng cú pháp như sau
 - `<chuỗi>[vị trí A : vị trí B]`
 - `<chuỗi>[vị trí A : vị trí B : bước nhảy]`
- Giải thích:
 - Tạo chuỗi con bắt đầu từ <vị-trí-A> đến trước <vị-trí-B>
 - Tức là chuỗi con sẽ không gồm vị trí B
 - Nếu không ghi <vị-trí-A> thì mặc định là lấy từ đầu
 - Nếu không ghi <vị-trí-B> thì mặc định là đến hết chuỗi
 - Nếu không ghi <bước-nhảy> thì mặc định bước là 1
 - Nếu <bước-nhảy> giá trị âm thì sẽ nhận chuỗi ngược lại

Cắt chuỗi

```
s = '0123456789'
print(s[3:6])           # 345
print(s[3:])            # 3456789
print(s[:6])            # 012345
print(s[-7:-4])         # 345
print(s[-4:-7])         #
print(s[-4:-7:-1])      # 654
print(s[:len(s)])       # 0123456789
print(s[:len(s)-1])     # 012345678
print(s[:])             # 0123456789
print(s[len(s)::-1])    # 9876543210
print(s[len(s)-1::-1])  # 9876543210
print(s[len(s)-2::-1])  # 876543210
```


Định dạng chuỗi

- Dùng toán tử %: <chuỗi> % (<các tham số>)
 - Bên trong <chuỗi> có các kí hiệu đánh dấu nơi đặt lần lượt các tham số
 - Nếu đánh dấu %s: thay thế bằng tham số dạng chuỗi
 - Nếu đánh dấu %d: thay thế bằng tham số dạng nguyên
 - Nếu đánh dấu %f: thay thế bằng tham số dạng thực
 - Có thể thêm tham số chỉ độ rộng của định dạng (xem ví dụ)

- Ví dụ:

```
"Chao %s, gio la %d gio" % ('txnam', 10)
```

```
"Can bac 2 cua 2 = %f" % (2**0.5)
```

```
"Can bac 2 cua 2 = %10.3f" % (2**0.5)
```

```
"Can bac 2 cua 2 = %10f" % (2**0.5)
```

```
"Can bac 2 cua 2 = %.7f" % (2**0.5)
```

Định dạng chuỗi

- Python cho phép định dạng chuỗi ở dạng f-string

```
myname = 'DHTL'
s = f'This is {myname}.'           # 'This is DHTL.'
w = f'{s} {myname}'               # 'This is DHTL. DHTL'
z = f'{{s}} {s}'                  # '{s} This is DHTL.'
```

- Mạnh mẽ nhất là định dạng bằng format

```
# điền lần lượt từng giá trị vào giữa cặp ngoặc nhọn
'a: {}, b: {}, c: {}'.format(1, 2, 3)
# điền nhưng không lần lượt
'a: {1}, b: {2}, c: {0}'.format('one', 'two', 'three')
'two same values: {0}, {0}'.format(1, 2)
# điền và chỉ định từng giá trị
'1: {one}, 2: {two}'.format(one=111, two=222)
```

Định dạng chuỗi

■ Định dạng bằng format cho phép căn lề phong phú

căn giữa: ' aaaa ' '{:^10}'.format('aaaa')

căn lề trái: 'aaaa ' '{:<10}'.format('aaaa')

căn lề phải ' aaaa'

'{:>10}'.format('aaaa')

căn lề phải, thay khoảng trắng bằng -: '-----aaaa'

'{:>10}'.format('aaaa')

căn lề trái, thay khoảng trắng *: 'aaa*****'

'{:*<10}'.format('aaaa')

căn giữa, thay khoảng trắng bằng +: '+++aaaa+++'

'{:+^10}'.format('aaaa')

Các phương thức của chuỗi

Cú pháp: *object* . *method name* (*parameter list*)

- Các phương thức chỉnh dạng
 - `capitalize()`: viết hoa chữ cái đầu, còn lại viết thường
 - `upper()`: chuyển hết thành chữ hoa
 - `lower()`: chuyển hết thành chữ thường
 - `swapcase()`: chữ thường thành hoa và ngược lại
 - `title()`: chữ đầu của mỗi từ viết hoa, còn lại viết thường
- Các phương thức căn lề
 - `center(width [,fillchar])`: căn lề giữa với độ dài width
 - `rjust(width [,fillchar])`: căn lề phải
 - `ljust(width [,fillchar])`: căn lề trái

Các phương thức của chuỗi

- Các phương thức cắt phần dư
 - `strip([chars])`: loại bỏ những ký tự đầu hoặc cuối chuỗi thuộc vào danh sách [chars], hoặc ký tự trống
 - `rstrip([chars])`: làm việc như strip nhưng cho bên phải
 - `lstrip([chars])`: làm việc như strip nhưng cho bên trái
- Tách chuỗi
 - `split(sep, maxsplit)`: tách chuỗi thành một danh sách, sử dụng dấu ngăn cách sep, thực hiện tối đa maxsplit lần
 - Tách các số nhập vào từ một dòng: `input("Test: ").split(',')`
 - `rsplit(sep, maxsplit)`: thực hiện như split nhưng theo hướng ngược từ phía cuối chuỗi

Các phương thức của chuỗi

■ Các phương thức khác

- `join(list)`: ghép các phần tử trong list bởi phần gạch nối là nội dung của chuỗi, ví dụ: `'-'.join(('1', '2', '3'))`
- `replace(old, new [,count])`: thế nội dung old bằng nội dung new, tối đa count lần
- `count(sub, [start, [end]])`: đếm số lần xuất hiện của sub
- `startswith(prefix)`: kiểm tra đầu có là prefix không
- `endswith(prefix)`: kiểm tra cuối có là prefix không
- `find(sub[, start[, end]])`: tìm vị trí của sub (-1: không có)
- `rfind(sub[, start[, end]])`: như find nhưng tìm từ cuối
- `islower()`, `isupper()`, `istitle()`, `isdigit()`, `isspace()`, `isalpha()`, `isnumeric()`
- `index(sub)` giống find, nhưng sinh ngoại lệ nếu không tìm thấy

Các hàm dựng sẵn hỗ trợ chuyển đổi

- Hàm `chr(n)`: chuyển đổi giá trị số sang mã unicode
- Hàm `ord(c)`: chuyển đổi kí tự unicode sang giá trị số
- Hàm `len(s)`: trả về độ dài (số kí tự) của chuỗi
- Hàm `str(v)`: chuyển đổi giá trị của biến `v` sang thể hiện ở dạng chuỗi
- Ví dụ:

```
s = '€'          # s chứa kí tự euro có mã 8364
print(ord(s))    # in ra mã của s (8364)
print(chr(8364)) # in ra kí tự ứng với mã 8364
n = 3+4j         # n là số phức 3+4i
print(n)         # tự động chuyển n sang chuỗi và in ra
print(len(n))    # LỖI
print(len(str(n))) # in ra độ dài của n dạng chuỗi
```

Bài tập về xử lý chuỗi

1. Nhập một chuỗi từ bàn phím, kiểm tra xem nó có tận cùng bởi 3 dấu chấm than hay không (!!!), nếu không thì hãy thêm dấu chấm than vào cuối để chuỗi có tận cùng là 3 dấu chấm than.
2. Nhập dãy số từ bàn phím (các số được gõ trên cùng một dòng, cách nhau bởi dấu cách hoặc tab), in ra dãy số vừa nhập.
3. Nhập một tên người từ bàn phím, hãy tách phần họ và tên riêng của người đó và in chúng ra màn hình (giả thiết họ và tên riêng chỉ gồm một âm).
4. Nhập một chuỗi từ bàn phím, hãy loại bỏ tất cả các chữ số khỏi chuỗi và in lại nội dung chuỗi mới ra màn hình.

Bài tập về xử lý chuỗi

5. Nhập một dãy các từ từ bàn phím, hãy in ra từ dài nhất trong dãy vừa nhập, in ra mọi từ có cùng độ dài nhất.
6. Nhập một dãy các từ từ bàn phím, thống kê xem có những chữ cái nào xuất hiện trong dãy và mỗi chữ xuất hiện bao nhiêu lần?
7. Nhập chuỗi S gồm toàn chữ số và số nguyên N, chỉ ra cách xóa đúng N kí tự khỏi S để được số có giá trị lớn nhất.
8. Nhập chuỗi S, hãy thay thế tất cả các chữ số trong S bằng kí tự hỏi chấm (?), sau đó in lại S ra màn hình
9. Nhập chuỗi S, kiểm tra xem chuỗi S có là dạng đối xứng hay không?

Phần 4

List (danh sách)

Giới thiệu và khai báo

- List = dãy các đối tượng (một loại array đa năng)
- Các phần tử con trong list không nhất thiết phải cùng kiểu dữ liệu
- Khai báo trực tiếp: liệt kê các phần tử con đặt trong cặp ngoặc vuông (`[]`), ngăn cách bởi dấu phẩy (,)

<code>[1, 2, 3, 4, 5]</code>	<code># list 5 số nguyên</code>
<code>['a', 'b', 'c', 'd']</code>	<code># list 4 chuỗi</code>
<code>[[1, 2], [3, 4]]</code>	<code># list 2 list con</code>
<code>[1, 'one', [2, 'two']]</code>	<code># list hỗn hợp</code>
<code>[]</code>	<code># list rỗng</code>

- Kiểu chuỗi (str) trong python có thể xem như một list đặc biệt, bên trong gồm toàn các str độ dài 1

Khởi tạo list

- Tạo list bằng constructor (hàm tạo)

```
l1 = list([1, 2, 3, 4])    # list 4 số nguyên
l2 = list('abc')           # list 3 chuỗi con
l3 = list()                # list rỗng
```

- Tạo list bằng **list comprehension** (bộ suy diễn danh sách)
một đoạn mã ngắn trả về các phần tử thuộc list

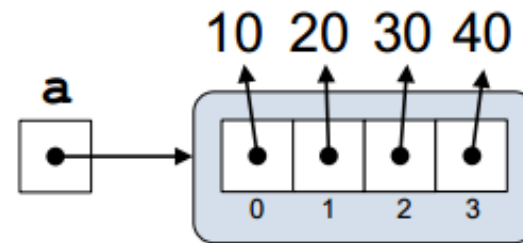
```
# list 1000 số nguyên từ 0 đến 999
X = [n for n in range(1000)]
# list gồm 10 list con là các cặp [x,
x2] # với x chạy từ 0 đến 9
Y = [[x, x*x] for x in range(10)]
```

Gán giá trị cho các phần tử trong list

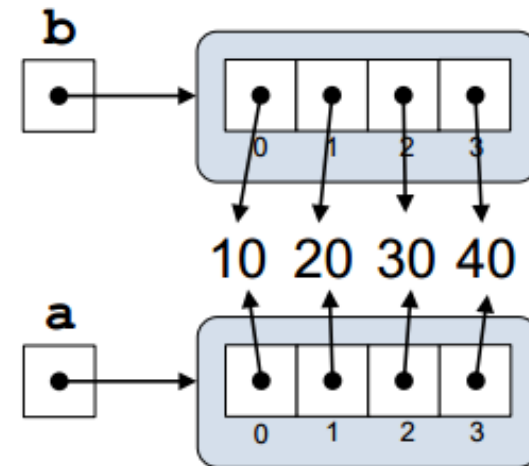
lst = [2, 4, 6, 8] → lst tham chiếu tới List

lst[2] → tham chiếu tới phần tử thứ 2 (giá trị = 6)

a = [10, 20, 30, 40]



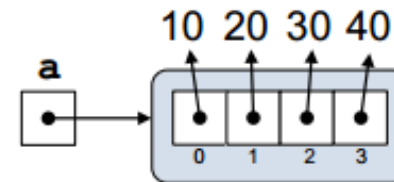
b = [10, 20, 30, 40]



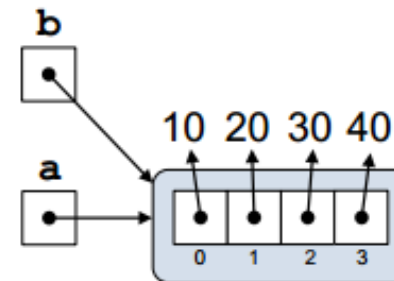
Gán giá trị cho các phần tử trong list

Ví dụ gán tham chiếu:

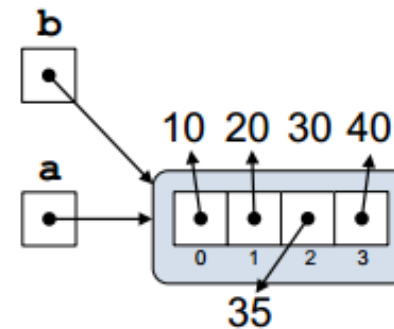
`a = [10, 20, 30, 40]`



`b = a`



`b[2] = 35`



So sánh 2 list: theo thứ tự từ điển (như str)

```
a = [1, 2, 3]
b = [1, 2, 3, 4]
c = [1.5]
d = ['a', 'b', 'c']
print(a > b)           # False
print(a == b)          # False
print(a < b)           # True
print(a + [4] == b)    # True
print(c <= b)          # False
print(d != a)          # True
print(d == list('abc')) # True
print(d >= c)          # Lỗi
```

Phép toán, chỉ mục và cắt

- Giữa list và str có sự tương đồng nhất định
 - List cũng hỗ trợ 3 phép toán: ghép nối (+), nhân bản (*) và kiểm tra nội dung (in)
 - List sử dụng hệ thống chỉ mục và các phép cắt phần con tương tự như str

- Điểm khác biệt: nội dung của list có thể thay đổi

khởi tạo list ban đầu

```
l1 = list([1, 2, 3, 4])
```

thay đổi giá trị của phần tử cuối cùng

```
l1[-1] = list('abc')
```

in nội dung list: [1, 2, 3, ['a', 'b', 'c']]

```
print(l1)
```


Chỉ mục, lát cắt, xóa dữ liệu với list

```
a = list('abcde')
print(a)                # ['a', 'b', 'c', 'd', 'e']
a[-1] = [0, 5, 9]       # thay đổi phần tử cuối cùng
print(a)                # ['a', 'b', 'c', 'd', [0, 5, 9]]
a[1:3] = [1, 2, 3]      # thay đổi một đoạn
print(a)                # ['a', 1, 2, 3, 'd', [0, 5, 9]]
print(a[2::-1])         # [2, 1, 'a']
a[2::-1] = [0]          # lỗi, đoạn ngược không thể thay đổi
del a[2::-1]            # xóa đoạn con từ 2 trở về đầu
print(a)                # [3, 'd', [0, 5, 9]]
del a                   # xóa biến a
print(a)                # lỗi, a không tồn tại
```

Các phương thức của list

- Một số phương thức thường hay sử dụng
 - `count(sub, [start, [end]])`: đếm số lần xuất hiện của sub
 - `index(sub[, start[, end]])`: tìm vị trí xuất hiện của sub, phát sinh lỗi `ValueError` nếu không tìm thấy
 - `clear()`: xóa trắng list
 - `append(x)`: thêm x vào cuối list
 - `extend(x)`: thêm các phần tử của x vào cuối list
 - `insert(p, x)`: chèn x vào vị trí p trong list
 - `pop(p)`: bỏ phần tử thứ p ra khỏi list (trả về giá trị của phần tử đó), nếu không chỉ định p thì lấy phần tử cuối

Các phương thức của list

- Một số phương thức thường hay sử dụng
 - `copy()`: tạo bản sao của list (tương tự `list[:]`)
 - `remove(x)`: bỏ phần tử đầu tiên trong list có giá trị x, báo lỗi `ValueError` nếu không tìm thấy
 - `reverse()`: đảo ngược các phần tử trong list
 - `sort(key=None, reverse=False)`: mặc định là sắp xếp các phần tử từ bé đến lớn trong list bằng cách so sánh trực tiếp giá trị

```
x = "Trương Xuân Nam".split()
x.sort(key=str.lower)
print(x)
```

Ví dụ về sắp xếp với list

```
a = ['123', '13', '90', "-1", -100]
a.sort()                # lỗi, vì các phần tử không cùng kiểu
a[-1] = '-100'         # thay đổi phần tử cuối thành dạng chuỗi
a.sort()               # xếp tăng dần theo so sánh chuỗi
print(a)               # ['-1', '-100', '123', '13', '90']
a.sort(key=int)        # chuyển thành số nguyên rồi sắp xếp
print(a)               # ['-100', '-1', '13', '90', '123']
a.sort(key=int, reverse=True)
print(a)               # ['123', '90', '13', '-1', '-100']
```

Duyệt list với vòng lặp for

```
my_list = ['foo', 'bar', 'baz', 'noo']
```

```
# duyệt các phần tử, cách làm đơn giản nhất
```

```
for item in my_list:  
    print(item)
```

```
# duyệt các phần tử theo giá trị chỉ mục
```

```
for i in range(len(my_list)):  
    print(i, my_list[i])
```

```
# duyệt theo giá trị chỉ mục nhưng nhanh hơn
```

```
# với cách sử dụng hàm enumerate
```

```
for i, item in enumerate(my_list):  
    print(i, my_list[i])
```

Ví dụ về làm việc với list

tạo và in nội dung của một list

```
danhsach = ["apple", "banana", "cherry"]
```

```
for x in danhsach:
```

```
    print(x)
```

thêm một phần tử vào cuối list và lại in ra

```
danhsach.append("orange")
```

```
print(danhsach)
```

thêm một phần tử vào giữa list và in ra

```
danhsach.insert(1, "orange")
```

```
print(danhsach)
```

sắp xếp lại danh sách và in ra

```
danhsach.sort()
```

```
print(danhsach)
```

Một số thao tác thông dụng với list

1. Tạo một list
2. Duyệt list
3. Truy cập phần tử theo chỉ số
4. Thay đổi nội dung phần tử
5. Cắt list
6. Thêm phần tử vào list
7. Bỏ phần tử khỏi list
8. Tìm kiếm phần tử trong list
9. Sắp xếp các phần tử trong list
10. List lồng ghép

List nhiều chiều

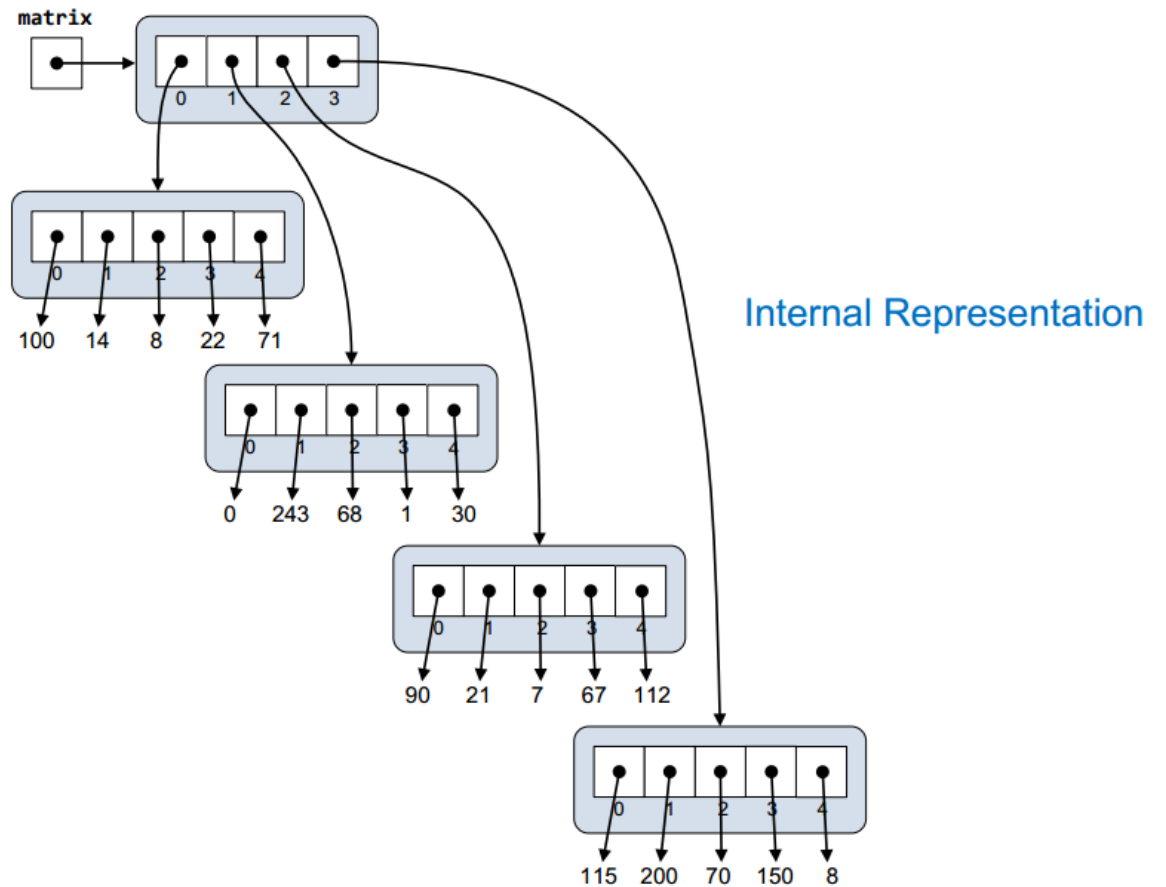
Duyệt list đa chiều:

```
matrix =  
    [100, 14, 8, 22, 71],  
    [ 0, 243, 68, 1, 30],  
    [ 90, 21, 7, 67, 112],  
    [115, 200, 70, 150, 8]  
]  
print(matrix)  
  
for row in matrix: # duyệt từng dòng  
    for elem in row: # Lấy từng phần tử trên mỗi dòng  
        print('{:>4}'.format(elem), end=' ')  
    print()
```


List nhiều chiều

matrix

	0	1	2	3	4
0	100	14	8	22	71
1	0	243	68	1	30
2	90	21	7	67	112
3	115	200	70	150	8



Phần 5

Tuple (hàng)

Bất biến (immutable) và Khả biến (mutable)

- Bất biến = không thay đổi, các loại dữ liệu bất biến thông dụng trong Python: `bool`, `int`, `float`, `str`, `tuple` và `frozenset`
- Khả biến = có thể thay đổi, các loại dữ liệu khả biến thông dụng trong Python gồm: `list`, `set`, `dict`
- Chúng ta vẫn thay đổi giá trị của `int`, tại sao nói “bất biến”
 - Python không thực sự thay đổi giá trị của `int`, phần mềm tạo vùng nhớ chứa giá trị mới và cho biến “trở” tới vùng đó
- Ví dụ để hiểu rõ cơ chế này:

```
n = 100
print(n, id(n))      # 100 và id của biến n
n = n + 1
print(n, id(n))      # 101 và id của n thay đổi so với trên
```

Tuple là một dạng readonly list (immutable)

- Tuple = dãy các đối tượng (list), nhưng không thể bị thay đổi giá trị trong quá trình tính toán
- Như vậy str giống tuple nhiều hơn list
- Khai báo trực tiếp bằng cách liệt kê các phần tử con đặt trong cặp ngoặc tròn (), ngăn cách bởi phẩy

(1, 2, 3, 4, 5) # tuple 5 số nguyên

('a', 'b', 'c', 'd') # tuple 4 chuỗi

(1, 'one', [2, 'two']) # Tuple hỗn hợp

(1,) # Tuple 1 phần tử

() # tuple rỗng

Khai báo tuple không nhất thiết phải dùng ()

```
t0 = 'a', 'b', 'c'           # tuple 3 chuỗi
t1 = 1, 2, 3, 4, 5           # tuple 5 số nguyên
t2 = ('a', 'b', 'c')         # tuple 3 chuỗi
t3 = (1, 'one', [2, 'two'])   # tuple hỗn hợp
t4 = 1, 'one', [2, 'two']     # tuple hỗn hợp
t5 = ()                      # tuple rỗng
t6 = 'a',                    # tuple một phần tử
t7 = ('a')                   # KHÔNG phải tuple (là chuỗi)
t8 = ('a',)                  # tuple một phần tử
t9 = ('a', )                 # tuple, không đúng chuẩn python
```

Tuple và list nhiều điểm giống nhau

- Tuple có thể tạo bằng constructor hoặc generator (bộ sinh) – một cách viết tương tự list comprehension
- Tuple hỗ trợ 3 phép toán: +, *, in
- Tuple cho phép sử dụng chỉ mục và cắt
- Các phương thức thường dùng của tuple
 - `count(v)`: đếm số lần xuất hiện của v trong tuple
 - `index(sub[, start[, end]])`: tương tự như str và list
- Tuple khác list ở điểm nào?
 - Chiếm ít bộ nhớ hơn
 - Nhanh hơn

Bô sinh của tuple chỉ dùng được 1 lần

```
# t0 viết giống như bộ suy diễn danh sách
t0 = (c for c in 'Hello!')
# tạo t1 từ t0
t1 = tuple(t0)
# tạo t2 từ t0
t2 = tuple(t0)

print(t0)    # <generator object <genexpr> at XXXX>
print(t1)    # ('H', 'e', 'l', 'l', 'o', '!')
print(t2)    # () <~~ như vậy t0 chỉ dùng được một lần
```

Tính bất biến của kiểu tuple

```
a = (1, 2, [3, 4], 'abc')
print(a[2])                # [3, 4]
a[2].append('xyz')         # a[2] trở thành [3, 4, 'xyz']
print(a)                   # (1, 2, [3, 4, 'xyz'], 'abc')
a[2] = '123'               # lỗi
a = (1, 2, 3)              # ok: thay đổi a thành tuple mới
a += (4, 5)                # ok: a thay đổi thành tuple mới
print(a)                   # (1, 2, 3, 4, 5)
del a                      # ok: xóa hoàn toàn a
```


Hàm dựng sẵn làm việc với list và tuple

- Hàm **all**(X): trả về True nếu tất cả các phần tử của X đều là True hoặc tương đương với True hoặc x rỗng
- Hàm **any**(X): trả về True nếu có ít nhất một phần tử của X là True hoặc tương đương với True
- Hàm **len**(X): trả về số lượng phần tử của X
- Hàm **list**(X): tạo một list gồm các phần tử con của X
- Hàm **max**(X): trả về giá trị lớn nhất trong X
- Hàm **min**(X): trả về giá trị nhỏ nhất trong X
- Hàm **sorted**(X): trả về danh sách mới gồm các phần tử của X đã được sắp xếp
- Hàm **sum**(X): trả về tổng các phần tử trong X

Phần 6

Range (miền)

Range là một tuple đặc biệt?

- Chúng ta đã làm quen với range khi dùng vòng for
 - `range(stop)`: tạo miền từ 0 đến stop-1
 - `range(start, stop[, step])`: tạo miền từ start đến stop-1, với bước nhảy là step
 - Nếu không chỉ định thì `step = 1`
 - Nếu `step` là số âm sẽ tạo miền đếm giảm dần (`start > stop`)
- Vậy range khác gì một tuple đặc biệt
 - Range chỉ chứa số nguyên
 - Range nhanh hơn rất nhiều
 - Range chiếm ít bộ nhớ hơn
 - Range vẫn hỗ trợ chỉ mục và cắt (nhưng khá đặc biệt)
 - Không nên coi Range là một Tuple!!!

Bài tập

1. Người dùng nhập từ bàn phím liên tiếp các từ tiếng Anh viết tách nhau bởi dấu cách. Hãy nhập chuỗi đầu vào và tách thành các từ sau đó in ra màn hình các từ đó theo thứ tự từ điển.
2. Người dùng nhập từ bàn phím chuỗi các số nhị phân viết liên tiếp được nối nhau bởi dấu phẩy. Hãy nhập chuỗi đầu vào sau đó in ra những giá trị được nhập.
3. Nhập số n , in ra màn hình các số nguyên dương nhỏ hơn n có tổng các ước số lớn hơn chính nó.
4. Nhập vào một chuỗi từ người dùng, kiểm tra xem đó có phải địa chỉ email hợp lệ hay không?

Bài tập

5. Nhập n , in n dòng đầu tiên của tam giác pascal
- 6.*Hãy nhập số nguyên n , tạo một list gồm các số fibonacci nhỏ hơn n và in ra
 - Dãy fibonacci là dãy số nguyên được định nghĩa một cách đệ quy như sau: $f(0)=0$, $f(1) = 1$, $f(1<n) = f(n-1) + f(n-2)$
7. Tạo tuple P gồm các số nguyên tố nhỏ hơn 1 triệu
 - Số nguyên tố là số tự nhiên có 2 ước số là 1 và chính nó.